

Locuto-Rio

Documento de Visão do Sistema

Machado dos Santos

2026-08-19

Sumário

Documento de Visão do Sistema	3
Resumo Executivo	3
Propósito Central e Proposta de Valor	3
Natureza e Funcionamento da Solução	3
Histórias de Utilizador	4
Apêndices Técnicos	5
Pressupostos Técnicos Fundamentais	5
Restrições Operacionais e de Engenharia	6
Estrutura Inicial de Componentes e Papéis	7
Diretrizes para a Engenharia de Requisitos Detalhada	7
Glossário	8

Documento de Visão do Sistema

LOCUTO-RIO: PLATAFORMA WEB OPEN-SOURCE DE PROTOTIPAGEM GUIADA POR MODELOS

O presente documento consolidou a formulação conceptual do projeto, tendo-se estruturado em duas partes fundamentais: um **resumo executivo**, centrado na demonstração de valor e nas narrativas de utilizador, e os respetivos **apêndices técnicos**, orientados para a estruturação operacional, as restrições do sistema e as diretrizes de arquitetura.

Resumo Executivo

Propósito Central e Proposta de Valor

O sistema consiste numa aplicação web de código aberto, orientada a modelos, concebida para dispensar a intervenção direta de designers na fase de prototipagem durante o levantamento de requisitos. Esta solução capacita os analistas de negócio e as equipas externas ao departamento de tecnologias de informação a produzir protótipos de alta-fidelidade e código estático de forma autónoma. Deste modo, elimina-se a necessidade de programar código-fonte ou de operar ferramentas técnicas de elevada complexidade.

Natureza e Funcionamento da Solução

A solução visa substituir o fluxo tradicional de desenho de esquemas visuais (*wireframes*) e a subsequente codificação de protótipos estáticos por interfaces orientadas por formulários específicos para cada tipo de artefacto. Para este efeito, prevê-se o desenvolvimento de um assistente dedicado à conceção de páginas institucionais e promocionais (*hotsites*), bem como de um assistente especializado na produção de formulários para operações de gestão de dados (criar, consultar, atualizar e remover registos) em aplicações institucionais.

A sustentabilidade e a eficácia da solução assentam na separação funcional de três intervenientes fundamentais no processo:

1. **A equipa de design** assume a governação visual e os requisitos de acessibilidade, sendo responsável pela criação e pela manutenção dos ficheiros de esquema de página e dos componentes de layout institucional.
2. **Os utilizadores sem competências técnicas** constroem, personalizam e configuram as páginas através de interfaces exclusivamente visuais e de assistentes orientados,

prescindindo de acesso direto ao código-fonte ou às especificidades das linguagens de formatação.

3. **As equipas de desenvolvimento** beneficiam da exportação direta de código de interface de utilizador limpo, estruturado e semântico, o que permite uma integração rápida e eficiente nos motores de processamento e lógica interna das aplicações da organização.

Histórias de Utilizador

As narrativas de utilizador (*user stories*) apresentadas em seguida operacionalizam os fluxos de trabalho e demonstram concretamente o valor gerado para os diferentes intervenientes na solução.

US01 – PARAMETRIZAÇÃO VISUAL AUTÓNOMA (ANALISTA DE NEGÓCIO / UTILIZADORES SEM COMPETÊNCIAS TÉCNICAS)	
Como	Analista de Negócios / Utilizador sem competências técnicas,
Quero	gerar protótipos de alta-fidelidade preenchendo assistentes guiados por formulários (ex.: assistente de formulários CRUD ou páginas institucionais),
Para que	possa validar regras de negócio e interfaces com os stakeholders sem escrever linhas de código nem utilizar ferramentas complexas.

US02 – GOVERNAÇÃO VISUAL E PADRONIZAÇÃO (DESIGNER DE UI/UX)	
Como	Designer responsável pelo <i>Design System</i> institucional,
Quero	criar e manter modelos padronizados em motores de <i>templating</i> ,
Para que	todas as interfaces geradas pela organização mantenham rigorosa conformidade visual, boas práticas de acessibilidade e governança estética centralizada.

US03 – EXPORTAÇÃO DE CÓDIGO SEMÂNTICO (ENGENHEIRO DE SOFTWARE)	
Como	Desenvolvedor de Software / Engenheiro de Backend,
Quero	exportar código UI estático limpo, semântico e estruturado a partir dos protótipos gerados,

Para que	possa integrá-los diretamente nas plataformas de backend (PHP, Java, JavaScript).
----------	---

Apêndices Técnicos

Pressupostos Técnicos Fundamentais

- **Paradigma Orientado a Modelos (*Template-Driven*) [Obrigatório]:** O sistema deve manter uma separação funcional estrita entre a camada de dados/parâmetros (fornecidos via formulários) e a camada de apresentação estética (gerida pelos ficheiros de layout).
 - *Justificação (Rationale):* Esta separação estrita garante que alterações visuais de layout efetuadas pela equipa de design não corrompam os dados preenchidos pelos analistas e utilizadores finais, permitindo também a atualização estética de *hotsites* e formulários retroativamente sem necessidade de reintroduzir dados.
- **Motor de Modelação (*Template Engine*) [Desejável]:** O compilador do sistema deveria utilizar inicialmente a tecnologia *Handlebars* para a renderização determinística e compilação dos artefactos visuais.
 - *Justificação (Rationale):* O *Handlebars* é uma tecnologia de compilação sem lógica complexa integrada nos modelos (*logicless templates*), o que simplifica a aprendizagem por parte da equipa de design de UI/UX e impede a introdução accidental de bugs de lógica na camada de apresentação.
- **Estrutura de Estilização e *Design System* [Desejável]:** O sistema deveria adotar inicialmente a framework *Bootstrap* como base para os seus componentes de estilo corporativos, assegurando responsividade nativa de ecrã e consistência de grelha visual.
 - *Justificação (Rationale):* A escolha do *Bootstrap* aproveita as competências técnicas já existentes no seio da equipa de desenvolvimento e garante consistência imediata de acessibilidade e compatibilidade em múltiplos navegadores web.
- **Interoperabilidade Multiplataforma de Saída [Obrigatório]:** O código de interface gerado pelo sistema deve ser agnóstico e pronto para integração imediata nos principais ambientes corporativos e plataformas de processamento interno da organização (como PHP, Java ou JavaScript).

- *Justificação (Rationale)*: Isto permite que o código exportado seja agnóstico a *Stacks* de backend específicas, evitando o bloqueio tecnológico (*vendor lock-in*) e permitindo que equipas de desenvolvimento distintas consumam a mesma interface limpa.

Restrições Operacionais e de Engenharia

- **Isolamento de Código para Utilizadores Finais [Obrigatório]**: A interface de negócios destinada aos analistas e utilizadores corporativos deve ocultar e impedir qualquer edição direta de *tags* HTML, folhas de estilo CSS ou marcações em *Markdown*.
 - *Justificação (Rationale)*: Esta restrição é crítica para evitar que utilizadores sem competências de engenharia de software ou de web design alterem elementos de formatação que possam quebrar a identidade visual institucional, corromper a compatibilidade de layout ou violar regulamentos de acessibilidade web.
- **Qualidade Semântica do Código de Saída [Obrigatório]**: O motor de compilação e exportação deve gerar código limpo e semântico, sendo proibida a introdução de *tags* redundantes, estilos em linha (*inline*) desnecessários ou código aninhado espúrio.
 - *Justificação (Rationale)*: O código gerado destina-se a ser integrado diretamente em sistemas corporativos de produção pelas equipas de engenharia. Código limpo elimina o padrão de “código sujo” comum em geradores WYSIWYG e facilita a manutenção, depuração e auditoria de segurança das aplicações.
- **Licenciamento e Governança [Obrigatório]**: O projeto do sistema deve reger-se sob um modelo de licença de Código Aberto (*Open Source*) não-recíproca (como MIT ou BSD).
 - *Justificação (Rationale)*: Uma licença permissiva e não-recíproca permite que o gerador seja integrado, estendido ou encapsulado em aplicações proprietárias internas da organização sem a obrigação legal de expor o código comercial de backend dessas soluções corporativas.

Estrutura Inicial de Componentes e Papéis

Perfil / Ator	Responsabilidade Operacional	Artefactos & Tecnologias Envolvidas
Design Team	Governança visual, acessibilidade, modelagem de layouts e regras do <i>design system</i> .	Ficheiros de modelo (<i>Handlebars</i>), Framework CSS (<i>Bootstrap</i>), Tokens Visuais.
Negócios & Marketing	Parametrização de conteúdo, composição de dados e definição de fluxos de ecrã via assistentes.	Assistentes Especializados (Páginas Institucionais, CRUD Académico), Formulários Visuais.
Desenvolvimento	Consumo do código UI limpo gerado e acoplamento com serviços, APIs e lógica de backend.	HTML5 Semântico, CSS3 Estruturado, <i>Stacks</i> de Backend (PHP, Java, JavaScript).

Diretrizes para a Engenharia de Requisitos Detalhada

Para as fases subsequentes de especificação e desenvolvimento de software, os futuros colaboradores devem priorizar:

1. **Catálogo de Assistentes (*Wizards*):** Mapeamento formal de todos os campos necessários para cada categoria de *hotsite* ou formulário de dados.
 - a. *Justificação (Rationale):* Necessário para estruturar o modelo de dados JSON que alimentará o motor de *templates*, servindo de base para o desenvolvimento das validações visuais das interfaces administrativas.
2. **Mecanismo de Validação Sintática e Semântica:** Implementação de testes automatizados no motor de compilação para certificar a validade W3C do código de interface gerado.
 - a. *Justificação (Rationale):* Essencial para assegurar de forma automatizada que nenhum modelo produzido por designers ou introdução de dados por utilizadores resulte em código HTML inválido que possa falhar em determinados navegadores ou leitores de ecrã para acessibilidade.
3. **Módulo de Exportação:** Especificação detalhada do pacote comprimido de ficheiros estáticos (HTML, CSS estruturado, JS e dependências locais) exportados pelo sistema.

- a. *Justificação (Rationale)*: Garante que o artefacto final possa ser introduzido diretamente e sem passos de adaptação manuais adicionais nos pipelines corporativos de integração e entrega contínuas (CI/CD).

Glossário

- **API (*Application Programming Interface* - Interface de Programação de Aplicações)**: Conjunto de regras e protocolos que permite que diferentes aplicações informáticas comuniquem e partilhem dados e funcionalidades de forma automática, segura e sem intervenção humana.
- **Acessibilidade Web**: O conjunto de práticas de design e desenvolvimento de software que garante que qualquer pessoa, independentemente das suas capacidades físicas ou cognitivas, consiga aceder, navegar e utilizar sítios e aplicações informáticas sem barreiras.
- **Bootstrap**: Uma das estruturas de código (frameworks) de estilização visual mais amplamente adotadas no desenvolvimento web. Permite criar componentes estéticos que se adaptam automaticamente a ecrãs de computadores, tablets e telemóveis (responsividade).
- **CI/CD (*Continuous Integration* / *Continuous Delivery* - Integração e Entrega Contínuas)**: Prática automatizada em engenharia de software onde as alterações ao código feitas pelos programadores são integradas, testadas e publicadas em produção automaticamente, poupando tempo e eliminando passos de verificação manuais.
- **CRUD (*Create, Read, Update, Delete* - Criar, Consultar, Atualizar, Remover)**: Sigla que sumariza as quatro operações básicas de manipulação de dados que o utilizador final pode executar numa base de dados.
- **Design System**: Um repositório centralizado de componentes visuais reutilizáveis, padrões de design e tokens estéticos (como cores e tipografia) que servem como a única fonte de verdade visual para todas as plataformas de uma organização.
- **Handlebars**: Motor de modelação informática (*template engine*) utilizado para carregar ficheiros estéticos e combiná-los deterministicamente com ficheiros de parâmetros ou dados, gerando a página final em formato legível.
- **Hotsite**: Um sítio na Internet focado numa campanha, produto, evento ou propósito específico de curta ou média duração, desenvolvido sob uma identidade visual própria.

- **HTML5 Semântico:** Linguagem utilizada para descrever a estrutura de páginas web que emprega etiquetas especiais para expressar explicitamente o propósito de cada componente (ex: usar <header> para cabeçalho, <nav> para navegação, etc.), facilitando a acessibilidade para leitores de ecrã e a indexação por parte de motores de pesquisa.
- **JSON (*JavaScript Object Notation*):** Formato padrão de ficheiro de texto, extremamente leve, estruturado e fácil de ler, utilizado universalmente na Web para enviar e receber dados entre sistemas informáticos.
- **Licença de Código Aberto Permissiva (ex: MIT ou BSD):** Tipo de licenciamento de software que concede direitos livres para usar, alterar e distribuir o software de forma privada ou integrada, sem impor a obrigação legal de tornar públicos os desenvolvimentos comerciais proprietários subsequentes.
- **Logicless Templates (Modelos Sem Lógica):** Ficheiros de visualização estética que apenas definem a disposição dos dados no ecrã sem misturar lógica de computação ou de processamento de backend, isolando a camada de design de eventuais bugs de sistema.
- **Markdown:** Linguagem de marcação muito leve que permite a formatação de textos (negritos, listas, títulos) através de caracteres comuns do teclado, amplamente utilizada na documentação e READMEs de software de código aberto.
- **Paradigma Orientado a Modelos (*Template-Driven*):** Padrão de engenharia de software em que os dados introduzidos pelo utilizador são mantidos de forma estritamente separada e isolada dos layouts de visualização estética, permitindo alterações de design sem impacto nos dados armazenados.
- **Tokens Visuais (*Design Tokens*):** Variáveis atómicas fundamentais (ex: definições exatas de espaçamentos, tipografias e paletas de cores corporativas) que alimentam e garantem a manutenção uniforme da identidade de marca em todas as plataformas informáticas da organização.
- **Vendor Lock-in (Bloqueio Tecnológico de Fornecedor):** Fenómeno em que uma organização se torna dependente de um único fornecedor, sistema ou *stack* de tecnologia específica devido às severas dificuldades técnicas e ao elevado custo financeiro de migração para outra plataforma.
- **W3C (*World Wide Web Consortium*):** Comunidade internacional que estabelece diretrizes e normas técnicas comuns de forma a garantir que todos os navegadores

web renderizem os elementos de HTML e CSS de forma perfeitamente idêntica e semântica.

- **WYSIWYG (*What You See Is What You Get* - O Que Vê É O Que Obtém):**
Editores de interface que geram código automaticamente à medida que o utilizador arrasta ou desenha os componentes visuais. Apesar de fáceis de operar, tendem a produzir código ineficiente, indevido e redundante que dificulta o backend (conhecido na indústria como “código sujo”).